

How to Design Database using EF Core Code First Approach:

In this article, I will discuss **How to Design a Database using the EF Core Code First** Approach. The Code-First approach in Entity Framework Core (EF Core) allows us to design the database starting from the C# domain classes rather than designing the database first and then generating the models. Once we create the model classes and DbContext class, EF Core will take responsibility for translating these model classes into database tables.

Let us develop a comprehensive Student Management System for an educational institution. We will also use this application as the base for the upcoming articles. We will work with the same application we have been using so far. So, let us proceed to create the Model classes. We will create all the Model classes within the Entities folder.

Student:

The Student model represents students within the institution, including personal details and academic information. So, create a class file named **Student.cs** within the **Entities** folder and copy and paste the following code.

```
namespace EFCoreCodeFirstDemo.Entities
{
    public class Student
    {
        public int StudentId { get; set; }
        public string FirstName { get; set; }
        public string LastName { get; set; }
        public string Gender { get; set; }
        public DateTime DateOfBirth { get; set; }
        public int BranchId { get; set; }
        public Branch Branch { get; set; }
        public Address Address { get; set; }
        public ICollection<Course> Courses { get; set; }
    }
}
```

Properties Description:

- **StudentId:** Primary Key. Unique identifier for each student.
- **FirstName:** First name of the student.
- **LastName:** Last name of the student.
- **Gender:** Gender of the student.
- **DateOfBirth:** Date of birth of the student.
- **BranchId:** Foreign Key. Links the student to a specific branch.
- **Branch:** The Branch navigation property represents the branch the student belongs to.
- **Address:** Navigation property for the student's address.
- **Courses:** A collection representing the many-to-many relationship between students and courses.

Relationships:

- **Branch:** A Student belongs to one Branch (one-to-one).
- **Courses:** A Student can enroll in many Courses (one-to-many).
- **Address:** A Student has one Address (one-to-one relationship).

Branch:

This model represents a branch. That means there are multiple branches of the institutions. It will represent a particular branch. So, create a class file named **Branch.cs** within the **Entities** folder and copy and paste the following code.

```
namespace EFCoreCodeFirstDemo.Entities
{
    public class Branch
    {
        public int BranchId { get; set; }
        public string BranchLocation { get; set; }
        public string? BranchPhoneNumber { get; set; }
        public string? BranchEmail { get; set; }
        public ICollection<Student> Students { get; set; }
        public ICollection<Teacher> Teachers { get; set; }
    }
}
```

Properties Description:

- **BranchId:** Primary Key. Unique identifier for each branch.
- **BranchLocation:** Location or Address of the branch.
- **BranchPhoneNumber:** (Optional) Phone number for the branch.
- **BranchEmail:** (Optional) Email address for the branch.
- **Students:** Collection of students enrolled in this branch.
- **Teachers:** A collection of teachers has been assigned to this branch.

Relationships:

- **Students:** A Branch can have multiple Students (one-to-many).
- **Teachers:** A Branch can have multiple Teachers (one-to-many).

Teacher:

This model represents teachers in the system. So, create a class file named **Teacher.cs** within the **Entities** folder and copy and paste the following code.

```
namespace EFCoreCodeFirstDemo.Entities
{
    public class Teacher
    {
        public int TeacherId { get; set; }
        public string TeacherName { get; set; }
        public int BranchId { get; set; }
        public Branch Branch { get; set; }
        public Address Address { get; set; }
        public ICollection<Subject> Subjects { get; set; }
    }
}
```

Properties Description:

- **TeacherId:** Primary Key. Unique identifier for each teacher.
- **TeacherName:** Name of the teacher.
- **BranchId:** Foreign Key. Links the teacher to a specific branch.
- **Branch:** The Branch navigation property represents the branch the teacher works for.
- **Address:** Navigation property for the teacher's address.
- **Subjects:** Collection of subjects taught by the teacher.

Relationships:

- **Branch:** A Teacher works in one Branch (one-to-one).
- **Subjects:** A Teacher can teach multiple Subjects (one-to-many).
- **Address:** A Teacher has one Address (one-to-one relationship).

Subject:

This Model contains information about subjects taught in the institution, including their names and descriptions. So, create a class file named **Subject.cs** within the **Entities** folder and copy and paste the following code.

```
namespace EFCoreCodeFirstDemo.Entities
{
    public class Subject
    {
        public int SubjectId { get; set; }
        public string SubjectName { get; set; }
        public string Description { get; set; }
        public ICollection<Teacher> Teachers { get; set; }
        public ICollection<Course> Courses { get; set; }
    }
}
```

Properties Description:

- **SubjectId:** Primary Key. Unique identifier for each subject.
- **SubjectName:** Name of the subject.
- **Description:** Description of the subject.
- **Teachers:** Collection of teachers who teach the subject.
- **Courses:** Collection of courses that include the subject.

Relationships:

- **Teachers:** A Subject can have multiple Teachers (one-to-many).
- **Courses:** A Subject can be part of multiple Courses (one-to-many).

Address:

This model represents the addresses of both students and teachers. So, create a class file named **Address.cs** within the **Entities** folder and copy and paste the following code.

```

namespace EFCoreCodeFirstDemo.Entities
{
    public class Address
    {
        public int AddressId { get; set; }
        public string Street { get; set; }
        public string City { get; set; }
        public string State { get; set; }
        public string PostalCode { get; set; }
        public int? StudentId { get; set; }
        public Student Student { get; set; }
        public int? TeacherId { get; set; }
        public Teacher Teacher { get; set; }
    }
}

```

Properties Description:

- **AddressId:** Primary Key. Unique identifier for each address.
- **Street:** Street of the address.
- **City:** City of the address.
- **State:** State of the address.
- **PostalCode:** Postal code of the address.
- **StudentId:** Foreign Key (nullable). Links the address to a student (if applicable).
- **Student:** Navigation property is for the student who lives at this address.
- **TeacherId:** Foreign Key (nullable). Links the address to a teacher (if applicable).
- **Teacher:** The navigation property is for the teacher at this address.

Relationships:

- **Student:** An address can be assigned to one Student (one-to-one).
- **Teacher:** An address can be assigned to one Teacher (one-to-one).

Course:

This model represents courses that students enroll in. So, create a class file named **Course.cs** within the **Entities** folder and copy and paste the following code.

```

namespace EFCoreCodeFirstDemo.Entities
{
    public class Course
    {
        public int CourseId { get; set; }
        public string Name { get; set; }
        public string Description { get; set; }
        public string Fees { get; set; }
        public ICollection<Student> Students { get; set; }
        public ICollection<Subject> Subjects { get; set; }
    }
}

```

Properties Description:

- **CourseId:** Primary Key. Unique identifier for each course.
- **Name:** Name of the course.
- **Description:** Description of the course.
- **Fees:** Course fees.
- **Students:** Collection of students enrolled in the course.
- **Subjects:** Collection of subjects included in the course.

Relationships:

- **Students:** A Course can have multiple Students (one-to-many).
- **Subjects:** A Course can have multiple Subjects (one-to-many).

Modifying the Context Class:

Next, modify the **EFCoreDbContext** class as follows. Now, we will work with the StudentDB. If the StudentDB database exists, it will use it or create a new StudentDB database if it does not exist. Inside this StudentDB database, it will create the Required database tables, which we specified as DbSet<T> entities.

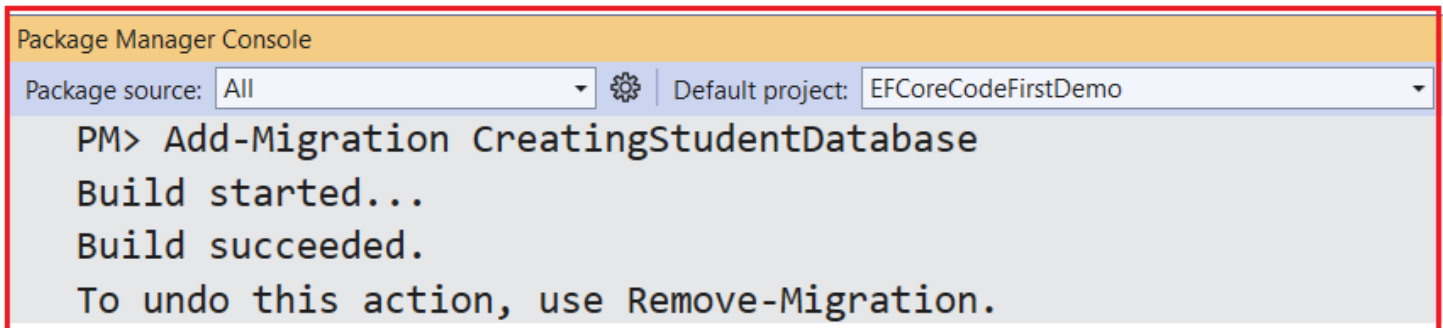
```
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Logging;
namespace EFCoreCodeFirstDemo.Entities
{
    public class EFCoreDbContext : DbContext
    {
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            // Enable Logging
            optionsBuilder.LogTo(Console.WriteLine, LogLevel.Information);
            //Connection String
            optionsBuilder.UseSqlServer("Server=LAPTOP-6P5NK25R\\SQLSERVER2022DEV;Database=StudentDB;Trusted_Connection=True;TrustServerCertificate=True");
            // DbSet properties represent the tables in the database.
            // Each DbSet corresponds to a table, and the type parameter corresponds to that table.
            public DbSet<Student> Students { get; set; }
            public DbSet<Teacher> Teachers { get; set; }
            public DbSet<Branch> Branches { get; set; }
            public DbSet<Address> Addresses { get; set; }
            public DbSet<Subject> Subjects { get; set; }
            public DbSet<Course> Courses { get; set; }
        }
    }
}
```

Generating Migration and Syncing with Database

The most important point is that whenever we add or update domain classes or configurations, we need to sync the database with the model. For every change we make to our Model classes, we must create a Migration File and sync

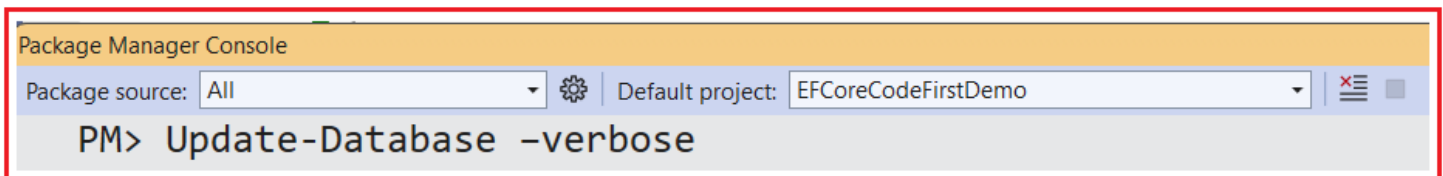
the database using the **Add-Migration** command and **Update-Database** commands using the Package Manager Console in Visual Studio.

To do so, open the NuGet Package Manager Console in Visual Studio by selecting **Tools => NuGet Package Manager => Package Manager Console** from the menu below. Then, execute the **Add-Migration CreatingStudentDatabase** command, as shown in the image below. Please select the Project where you want to generate the Migration.

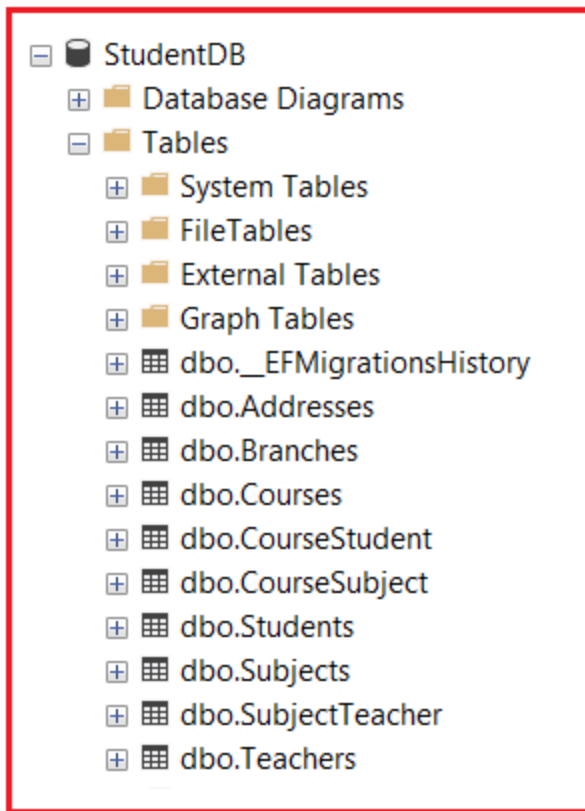


Updating the Database:

After creating the migration file, we need to update the database using the **Update-Database** command. We can use the **-verbose** option to view the generated SQL statements executed in the target database. So, open the Package Manager Console and execute the **Update-Database -Verbose** command, as shown in the image below.



Once the above command is executed successfully, it will generate and execute the required SQL Statements in the defined database. You can also verify the database and database tables using SSMS, as shown in the image below.



As you can see, in addition to the Addresses, Branches, Courses, Students, Subjects, and Teachers database tables that we specified as the DbSet property in our DbContext class, it also created the CourseStudent, CourseSubject, and SubjectTeacher tables.

The extra tables (CourseStudent, CourseSubject, and SubjectTeacher) that were generated during the migration process are junction tables (also called join tables) created by Entity Framework Core (EF Core) to manage the many-to-many relationships between the entities.

Why were these junction tables created?

Many-to-many relationships between two tables cannot be directly represented in relational databases. Instead, an intermediary table (also known as a junction table) is used to map the relationship. This table contains the foreign keys referencing the primary keys of the related tables. Let's understand the many-to-many relationships in our models and the corresponding junction tables:

Student ↔ Course (Many-to-Many)

Our Student class has an `ICollection<Course> Courses` property, indicating that a student can enroll in multiple courses. Similarly, our Course class has an `ICollection<Student> Students` property, indicating that a course can have multiple students.

Since EF Core needs to model this many-to-many relationship, it creates a junction table named **CourseStudent** (EF Core uses the names of both entities to name the junction table). This table stores:

- A foreign key to the Student (e.g., `StudentsStudentId`). It is created as the database name plus the primary key column name.
- A foreign key to the Course (e.g., `CoursesCourseId`). It is created as the database name plus the primary key column name.

This allows multiple students to enroll in multiple courses. If you verify the database, you will see the following structure of the CourseStudent table. It also creates the primary key as a composite primary key.

dbo.CourseStudent
Columns
CoursesCourseId (PK, FK, int, not null)
StudentsStudentId (PK, FK, int, not null)

Course ↔ Subject (Many-to-Many)

Our Course class has an `ICollection<Subject> Subjects` property, meaning a course can include multiple subjects. Similarly, our Subject class has an `ICollection<Course> Courses` property, meaning a subject can be part of multiple courses. So, to represent this many-to-many relationship, EF Core generates a junction table named `CourseSubject`, which contains:

- A foreign key to the Course (e.g., `CoursesCourseId`). It is pointing to the `CourseId` column of the `Courses` table.
- A foreign key to the Subject (e.g., `SubjectsSubjectId`). It is pointing to the `SubjectId` column of the `Subjects` table.

If you verify the database, you will see the following structure for the `CourseSubject` database table. It also creates the primary key as a composite primary key.

dbo.CourseSubject
Columns
CoursesCourseId (PK, FK, int, not null)
SubjectsSubjectId (PK, FK, int, not null)

Teacher ↔ Subject (Many-to-Many)

Our Teacher class has an `ICollection<Subject> Subjects` property, indicating that a teacher can teach multiple subjects. Similarly, in our Subject class, there's an `ICollection<Teacher> Teachers` property, indicating that multiple teachers can teach a subject. EF Core automatically creates a junction table named `SubjectTeacher` to represent this many-to-many relationship, storing:

- A foreign key to Teacher (e.g., `TeachersTeacherId`).
- A foreign key to Subject (e.g., `SubjectsSubjectId`).

If you verify the database, you will see the following structure for the `SubjectTeacher` table. It also creates the primary key as a composite primary key.

dbo.SubjectTeacher
Columns
SubjectsSubjectId (PK, FK, int, not null)
TeachersTeacherId (PK, FK, int, not null)

Note: In EF Core 5.0 and above, the framework automatically supports many-to-many relationships, so we don't need to define a separate entity for the junction table explicitly. EF Core automatically creates the junction tables to represent these relationships.

Adding Data for Testing Purposes:

In our upcoming articles, I will show you how to develop this application using ASP.NET Core MVC. But now, our intention is to understand the EF Core concepts like Lazy Loading, Eager Loading, Explicit Loading, Asynchronous Operations, Transaction Management, and Fluent API Configurations.

Let's first understand these concepts using these Models and the DbContext class. Once we understand the EF Core concepts, we will develop the complete application. To understand these concepts, we need some data. So, please execute the following SQL Statements in the StudentDB database we created using EF Core Migration to populate the database tables with data.

```
-- Insert data into Branches table
INSERT INTO Branches (BranchLocation, BranchPhoneNumber, BranchEmail)
VALUES
('BBSR', '212-555-1234', 'BBSR@dotnettutorials.net'),
('Delhi', '310-555-5678', 'Delhi@dotnettutorials.net'),
('Mumbai', '312-555-9876', 'Mumbai@dotnettutorials.net');
GO

-- Insert data into Courses table
INSERT INTO Courses (Name, Description, Fees)
VALUES
('Full Stack Dot Net', 'Comprehensive course covering front-end and back-end development using .NET technologies.', '6000'),
('Full Stack Java', 'End-to-end web development using Java technologies for front-end and back-end.', '5800'),
('Machine Learning', 'Introductory course to machine learning algorithms and data science.', '7000');
GO

-- Insert data into Subjects table
INSERT INTO Subjects (SubjectName, Description)
VALUES
('ASP.NET Core Development', 'Developing web applications using ASP.NET Core framework.'),
('C# Programming', 'Deep dive into C# language features and best practices.'),
('JavaScript Frameworks', 'Working with modern JavaScript frameworks such as Angular and React.'),
('Java Programming', 'Comprehensive coverage of Java programming language.'),
('Spring Boot Development', 'Creating REST APIs using Spring Boot framework in Java.'),
('Machine Learning Algorithms', 'Introduction to various machine learning algorithms and techniques.'),
('Python for Data Science', 'Learn Python programming with a focus on data analysis and manipulation.'),
('Neural Networks and Deep Learning', 'Introduction to neural networks and deep learning techniques.');
GO

-- Insert data into Students table
INSERT INTO Students (FirstName, LastName, Gender, DateOfBirth, BranchId)
VALUES
('John', 'Doe', 'Male', '2000-05-15', 1),
('Jane', 'Smith', 'Female', '2001-08-20', 2),
('Mark', 'Johnson', 'Male', '2002-03-10', 3),
('Emily', 'Clark', 'Female', '2000-11-05', 1),
```

```

('Sara', 'Taylor', 'Female', '2001-08-20', 2),
('James', 'Smith', 'Male', '2002-03-10', 3);
GO
-- Insert data into Teachers table
INSERT INTO Teachers (TeacherName, BranchId)
VALUES
('Dr. Alice Johnson', 1),
('Dr. Bob Brown', 2),
('Dr. Carol White', 3),
('Dr. David Green', 1);
GO
-- Insert data into Addresses table
-- Student Addresses
INSERT INTO Addresses (Street, City, State, PostalCode, StudentId)
VALUES
('123 Main St', 'New York', 'NY', '10001', 1),
('456 Oak St', 'Los Angeles', 'CA', '90001', 2),
('789 Pine St', 'Chicago', 'IL', '60601', 3),
('101 Maple St', 'New York', 'NY', '10001', 4);
GO
-- Teacher Addresses
INSERT INTO Addresses (Street, City, State, PostalCode, TeacherId)
VALUES
('221B Baker St', 'New York', 'NY', '10001', 1),
('742 Evergreen Terrace', 'Los Angeles', 'CA', '90001', 2),
('1600 Pennsylvania Ave', 'Chicago', 'IL', '60601', 3),
('10 Downing St', 'New York', 'NY', '10001', 4);
GO
-- Insert data into CourseSubject table (Mapping Courses to Subjects)
-- Full Stack Dot Net
INSERT INTO CourseSubject (CoursesCourseId, SubjectsSubjectId)
VALUES
(1, 1), -- Full Stack Dot Net -> ASP.NET Core Development
(1, 2), -- Full Stack Dot Net -> C# Programming
(1, 3); -- Full Stack Dot Net -> JavaScript Frameworks
GO
-- Full Stack Java
INSERT INTO CourseSubject (CoursesCourseId, SubjectsSubjectId)
VALUES
(2, 4), -- Full Stack Java -> Java Programming
(2, 5), -- Full Stack Java -> Spring Boot Development
(2, 3); -- Full Stack Java -> JavaScript Frameworks
GO
-- Machine Learning
INSERT INTO CourseSubject (CoursesCourseId, SubjectsSubjectId)
VALUES
(3, 6), -- Machine Learning -> Machine Learning Algorithms
(3, 7), -- Machine Learning -> Python for Data Science
(3, 8); -- Machine Learning -> Neural Networks and Deep Learning
GO
-- Insert data into CourseStudent table (Mapping Courses to Students)

```

```

INSERT INTO CourseStudent (CoursesCourseId, StudentsStudentId)
VALUES
(1, 1), -- John Doe -> Full Stack Dot Net
(2, 2), -- Jane Smith -> Full Stack Java
(3, 3), -- Mark Johnson -> Machine Learning
(2, 4), -- Emily Clark -> Full Stack Java
(1, 5), -- Sara Taylor -> Full Stack Dot Net
(3, 6); -- John Doe -> Machine Learning
GO

-- Insert data into SubjectTeacher table (Mapping Subjects to Teachers)
INSERT INTO SubjectTeacher (SubjectsSubjectId, TeachersTeacherId)
VALUES
(1, 1), -- ASP.NET Core Development -> Dr. Alice Johnson
(2, 1), -- C# Programming -> Dr. Alice Johnson
(3, 4), -- JavaScript Frameworks -> Dr. David Green
(4, 2), -- Java Programming -> Dr. Bob Brown
(5, 3), -- Spring Boot Development -> Dr. Carol White
(6, 4), -- Machine Learning Algorithms -> Dr. David Green
(7, 2), -- Python for Data Science -> Dr. Bob Brown
(8, 3); -- Neural Networks and Deep Learning -> Dr. Carol White
GO

```

So, using the EF Core Model First Approach, we have created the database and tables and populated them with some dummy data. In the next article, I will discuss [Lazy Loading, Eager Loading, and Explicit Loading](#) of Related Entities using EF Core. In this article, I explain How to Design a Database using the EF Core Code First Approach. I hope you enjoy this article on “How to Design Database using EF Core Code First Approach.”